MINI-PROJET POO · SUJET 1

Mobilité éco-responsable : le Campus Vert

Cahier des charges pédagogique

Wissem Zeddini · À lire avec le document « Mini-projets POO : cadre commun »

Client	Mme Ines Gharbi, responsable de la Mobilité du Campus Vert (client simulé)
Le logiciel	Gestion d'une flotte de véhicules électriques en libre-service : trottinettes, vélos et voitures
Difficulté	Intermédiaire (2 sur 3)
Notions du socle	Encapsulation, constructeurs, héritage, redéfinition, polymorphisme
Bonus possibles	Abstraction (badge Abstrait), exception personnalisée (badge Résilient), sauvegarde dans un fichier (badge Archiviste)
Livrable final	Code, fichier .jar, UML, rapport de 2 à 3 pages, démonstration de 5 minutes

1. Le contexte

Le **Campus Vert** (un campus fictif) accueille 8 000 étudiants répartis sur plusieurs bâtiments. Pour réduire la voiture individuelle, sa direction veut lancer une flotte de véhicules électriques en **libre-service** : on prend un véhicule à une station, on roule, on le repose.

Elle a besoin d'un logiciel qui gère les véhicules, leurs batteries, leurs locations et leurs pannes. **Vous êtes l'équipe de développement.** Le client est exigeant et il changera d'avis : concevez dès le départ pour que votre code puisse évoluer.

2. Le vocabulaire du métier

Terme	Sens pour le client
Véhicule	Un moyen de transport électrique de la flotte (trottinette, vélo, voiture).
Station	Un lieu de garage de véhicules, avec une capacité maximale.
Niveau de batterie	Le pourcentage de charge d'un véhicule, de 0 à 100.
Consommation	Le pourcentage de batterie perdu par kilomètre ; elle dépend du type de véhicule.
Verrouillé	Un véhicule garé à la station ; il est déverrouillé pendant un trajet.
Disponible	Un véhicule qu'on peut louer (voir la règle R2).
Trajet	Une location : un véhicule, une durée en minutes, une distance en kilomètres.
Tarif	Le prix d'un trajet, en millimes (1 dinar = 1 000 millimes).
Recette	La somme d'argent que rapportent les locations.
Panne	Une batterie qui tombe trop bas pendant un trajet : le véhicule s'arrête.
Maintenance	L'intervention qui remet un véhicule en état et recharge sa batterie.

3. Les règles du métier

Ces règles sont celles du client. Chacune doit être **protégée par votre code**, pas seulement respectée par vos tests.

N°	Règle	Apparaît en
R1	Le niveau de batterie reste **toujours entre 0 et 100 %**.	v0.2
R2	Un véhicule est **disponible** s'il est verrouillé, sans panne, avec **au moins 15 %** de batterie.	v0.2
R3	Chaque type de véhicule a son **tarif par minute**, en millimes (tableau ci-dessous).	v0.3
R4	Chaque type de véhicule consomme **son propre pourcentage** de batterie par kilomètre.	v0.3
R5	Dans une station, un **identifiant est unique**, et la capacité maximale ne peut pas être dépassée.	v0.3
R6	Si la batterie passe **sous 5 %** pendant un trajet, c'est la panne : le trajet s'arrête, rien n'est facturé, et l'alerte part vers l'équipe adaptée au type.	v1.0
R7	Le **tarif de nuit** est inférieur de 20 % au tarif de jour.	v1.0

Les types de véhicules

Type	Tarif (millimes/min)	Consommation (% par km)	Particularité	En cas de panne
Véhicule générique (classe mère)	100	2	—	Alerte simple
Trottinette	150	4	—	Une équipe la ramasse
Vélo électrique	100, plus 20 avec siège bébé	2	Siège bébé : oui ou non	Il part à l'atelier
Voiture partagée	400	1	Nombre de places ; permis requis	Remorquage

*Utilisez des **constantes nommées** plutôt que des nombres écrits au milieu du code. Les tarifs sont des **entiers** (millimes) : pas de nombres à virgule pour l'argent.*

4. Version v0.1 : Cadrage

Mail du client · v0.1

De : Ines Gharbi, Direction de la Mobilité, Campus Vert

Objet : Notre future flotte électrique

Bonjour,

Nous lançons une flotte de trottinettes, de vélos et de voitures électriques en libre-service. Avant de coder quoi que ce soit, je voudrais être sûre que vous avez compris mon métier. Montrez-moi les objets que vous avez repérés, ce que chacun sait de lui-même, et affichez-moi un premier véhicule à l'écran. Pas d'écran graphique : la console me suffit.

Cordialement.

N°	Demande du client	Notion
1.1	« En tant que responsable, je veux que vous identifiiez les objets de mon métier, afin que nous parlions le même langage. »	Analyse, UML
1.2	« En tant que responsable, je veux voir un véhicule à l'écran, afin de vérifier que mon besoin est compris. »	Classe, constructeur, main

Vous livrez

- Un **UML de départ** d'au moins 3 classes (par exemple Vehicule, Station et Demo), avec leurs attributs principaux.
- Des **squelettes de classes qui compilent**.
- Une classe **Demo** dont le main affiche un véhicule, par exemple : `V001 · Vélo Alpha · batterie 100.0 %`.

> **Indice**
> Un véhicule connaît au moins son **identifiant**, son **modèle** et son **niveau de batterie**. Ne cherchez pas encore les trois types : ils arrivent avec le mail de la version v0.3.

5. Version v0.2 : Objets fiables

> **Mail du client · v0.2**
> **De :** Ines Gharbi
> **Objet :** Votre première version : un problème de données
> Bonjour,
> Merci pour votre première version. J'ai fait un test : un technicien a tapé 120 % comme niveau de batterie, et le logiciel l'a accepté ! Mes données doivent rester cohérentes. Le niveau de batterie ne doit jamais sortir de 0 à 100 %, et personne ne doit pouvoir le modifier directement : seulement en rechargeant le véhicule ou en roulant.
> Je veux aussi savoir si un véhicule est louable, le créer neuf (batterie pleine) ou avec un niveau connu, et le lire en une ligne.
> Cordialement.

N°	Demande du client	Notion
2.1	« Je veux que le niveau de batterie reste toujours entre 0 et 100 %, afin d'éviter des chiffres absurdes. » (R1)	Encapsulation, validation
2.2	« Je veux recharger un véhicule d'un certain pourcentage sans dépasser 100 %, afin de refléter la réalité. »	Méthode métier
2.3	« Je veux enregistrer la consommation d'un trajet en kilomètres, sans que la batterie passe sous 0 %, afin de suivre l'usure. »	Méthode métier
2.4	« Je veux savoir si un véhicule est disponible, afin de ne louer que des véhicules utilisables. » (R2)	Méthode booléenne
2.5	« Je veux créer un véhicule neuf (batterie à 100 %) ou avec un niveau connu, afin de gagner du temps. »	Constructeurs surchargés
2.6	« Je veux lire un véhicule en une ligne, afin de contrôler vite. »	toString()

Vous livrez

- Tous les attributs **privés**, et **aucun setter pour la batterie** : seules les méthodes de recharge et de consommation la modifient.
- Deux constructeurs (véhicule neuf, véhicule avec niveau connu) qui **valident** le niveau reçu.
- Une **Demo** avec au moins trois véhicules, qui montre **un refus** (niveau de 120 %, recharge négative).

> **Indices**
> Testez les **valeurs limites** : -5, 0, 100 et 120. Les méthodes `Math.min` et `Math.max` gardent une valeur entre deux bornes. Pensez à valider aussi dans le constructeur : c'est la porte d'entrée la plus oubliée.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Niveau de batterie invalide (120.0 %) : batterie initialisée à 0 %.
V001 · Vélo Alpha · batterie 100.0 %
V002 · Scooter Bêta · batterie 40.0 %
V003 · Trottinette Gamma · batterie 0.0 %
Recharge invalide : le pourcentage doit être positif.
V001 · Vélo Alpha · batterie 80.0 %
V002 · Scooter Bêta · batterie 100.0 %
V001 disponible ? true
V003 disponible ? false
```

6. Version v0.3 : La famille

> **Mail du client · v0.3**
> **De :** Ines Gharbi
> **Objet :** Trois types de véhicules, et ce n'est pas fini
> Bonjour,
> Excellent travail. Le campus autorise maintenant trois types de véhicules : des trottinettes, des vélos et des voitures partagées. Chacun a son tarif par minute et consomme sa propre part de batterie (voir le tableau).
> Je veux regrouper les véhicules dans une station, tous types mélangés, les afficher d'un coup, et savoir ce que rapporterait une demi-heure d'utilisation de tous les véhicules disponibles.
> Et une chose importante : demain, j'ajouterai peut-être des motos électriques. **Je ne veux pas que vous réécriviez votre programme à chaque nouveau type.**
> Cordialement.

N°	Demande du client	Notion
3.1	« Je veux trois types de véhicules avec leurs particularités (siège bébé, nombre de places), afin de décrire fidèlement ma flotte. »	Héritage, super(...)
3.2	« Je veux que chaque type calcule son propre tarif et sa propre consommation, afin d'appliquer mes règles. » (R3, R4)	Redéfinition, @Override
3.3	« Je veux une station qui contient des véhicules de tous types, avec une capacité maximale et sans doublon d'identifiant, afin de refléter mes stations. » (R5)	Tableau ou liste du type commun
3.4	« Je veux afficher toute la flotte en une fois, afin de la contrôler. »	Une seule boucle, toString()

N°	Demande du client	Notion
3.5	« Je veux la recette théorique pour N minutes de tous les véhicules disponibles, afin de prévoir mon budget. »	Polymorphisme
3.6	« Je veux ajouter un nouveau type plus tard sans modifier la station, afin que mon logiciel évolue. »	Preuve du polymorphisme

Vous livrez

- Une **classe mère** qui porte ce qui est commun, et **trois classes filles** qui n'ajoutent que leurs particularités. L'UML est mis à jour.
- Une **Station** qui contient des véhicules de types différents, et **une seule boucle** pour les afficher et calculer la recette.
- La preuve que la station n'a pas besoin d'être modifiée pour accueillir un nouveau type.

> **Indices**
>
> Si vous écrivez **instanceof** dans la boucle, c'est que le polymorphisme manque : demandez-vous quelle méthode chaque type devrait redéfinir. Pour comparer deux identifiants (du texte), utilisez `equals`, jamais `==`.

Exemple de sortie d'une Demo (la vôtre peut différer) :

```
Identifiant T101 déjà présent : ajout refusé.
Station Bibliothèque (3/4)
  Trottinette T101 · Flash · batterie 100.0 %
  Vélo E201 · Eco · batterie 90.0 % · siège bébé
  Voiture C301 · Citadine · batterie 80.0 % · 4 places · permis requis
Disponibles : 3
Recette théorique, 30 min, de jour : 20100 millimes
Recette théorique, 30 min, de nuit : 16080 millimes
```

Les deux dernières lignes préparent la version suivante : le tarif de nuit.

7. Version v1.0 : Livraison

> **Mail du client · v1.0**
>
> **De :** Ines Gharbi
>
> **Objet :** Dernière étape avant le lancement
>
> Bonjour,
>
> Votre station fonctionne bien. Dernière étape avant le lancement. Mon cauchemar, c'est la **panne en pleine rue**. Je veux que votre logiciel simule un trajet kilomètre par kilomètre : si la batterie passe sous 5 %, le trajet s'arrête, rien n'est facturé, et l'alerte part vers la bonne équipe (ramassage d'une trottinette, atelier pour un vélo, remorquage d'une voiture).
>
> Je veux aussi un tarif de nuit inférieur de 20 %, un menu simple pour mes agents, et un fichier que je puisse lancer moi-même sur mon ordinateur.
>
> Cordialement.

N°	Demande du client	Notion
4.1	« Je veux simuler un trajet (véhicule, minutes, kilomètres), afin de tester mes règles avant le lancement. »	Assemblage

N°	Demande du client	Notion
4.2	« Je veux que la panne arrête le trajet, ne facture rien et déclenche l'alerte du bon type, afin de réagir vite. » (R6)	Défi métier, polymorphisme
4.3	« Je veux qu'un véhicule en panne ne soit plus louable jusqu'à sa maintenance, afin de ne pas louer un véhicule défaillant. »	État d'un objet
4.4	« Je veux un tarif de nuit à moins 20 %, sans erreur d'arrondi, afin d'encourager les trajets nocturnes. » (R7)	Calcul en entiers
4.5	« Je veux un menu console, afin que mes agents utilisent le logiciel sans toucher au code. »	Menu, boucle de saisie
4.6	« Je veux un fichier que je lance moi-même, afin de l'installer sur mon poste. »	Fichier .jar

Le défi métier : la panne en pleine rue

Pendant un trajet, après **chaque kilomètre**, la batterie baisse selon la consommation du type de véhicule. Dès qu'elle passe sous 5 % :

1. le trajet s'arrête et affiche à quel kilomètre la panne s'est produite ;

2. rien n'est facturé ;

3. le véhicule est déclaré **en panne** : il n'est plus disponible tant qu'une maintenance n'a pas eu lieu ;

4. l'**alerte est propre au type** : chaque type de véhicule réagit à sa façon, sans que la station demande « quel est ton type ? ».

Menu console attendu (exemple) :

```
=== CAMPUS VERT ===
1. Afficher la flotte
2. Louer et simuler un trajet
3. Recharger un véhicule
4. Recette théorique (jour ou nuit)
5. Maintenance d'un véhicule en panne
0. Quitter
Votre choix (0 à 5) :
```

Exemple de sortie d'une Demo de trajets (la vôtre peut différer) :

```
Véhicule C302 indisponible.
Départ de E202 (batterie 70.0 %), 10 km prévus.
Arrivée : 10 km, 25 min, 2500 millimes. Batterie restante : 50.0 %.
Départ de T102 (batterie 30.0 %), 8 km prévus.
Panne à 7 km sur 8 km prévus.
ALERTE MAINTENANCE : intervention requise sur T102
  → Une équipe ramasse la trottinette T102.
Trajet interrompu : rien n'est facturé.
Station Résidence (3/4)
  Trottinette T102 · Turbo · batterie 2.0 % · EN PANNE
  Voiture C302 · Berline · batterie 12.0 % · 5 places · permis requis
  Vélo E202 · Trek · batterie 50.0 %
Maintenance effectuée sur T102 : batterie à 100 %.
Départ de T102 (batterie 100.0 %), 8 km prévus.
Arrivée : 8 km, 15 min, 2250 millimes. Batterie restante : 68.0 %.
```

Indices

Le **tarif de nuit** : calculez en entiers (par exemple total × 80 / 100) ; multiplier par 0,8 introduit des nombres à virgule. L'alerte se déclenche par **une seule méthode** que chaque type redéfinit. Lisez bien l'**ordre** des vérifications : un véhicule indisponible ne doit jamais partir.

8. Les bonus du sujet

Badge	Défi
Abstrait	Rendez la classe mère **abstraite** : un « véhicule générique » ne se loue pas. Faites de `calculerTarif` une méthode abstraite. Que répond Java si vous écrivez `new Vehicule(...)` ? (« Vehicule is abstract; cannot be instantiated »). Justifiez votre choix dans le rapport.
Résilient	Créez une **exception personnalisée** (par exemple `BatterieFaibleException`) levée quand on tente de louer un véhicule sous le seuil, et interceptée par le menu.
Archiviste	Sauvegardez la flotte dans un fichier texte ou CSV à la fermeture (une ligne par véhicule : type, identifiant, modèle, batterie, particularité) et rechargez-la au démarrage.

9. Les pièges fréquents de ce sujet

- **Batterie qui dépasse 100 %** après une recharge : oublier de plafonner.
- **Comparer des identifiants avec ==** : cela compare les étiquettes, pas le texte. Utilisez `equals`.
- **Recopier recharger ou consommer dans chaque classe fille** : si vous copiez, c'est que la classe mère manque.
- **Oublier super(...)** dans le constructeur d'une fille, ou le placer après une autre instruction.
- **Un instanceof dans la boucle de la station** : le polymorphisme est contourné ; une nouvelle moto obligerait à modifier la station.
- **Le tarif de nuit en nombres à virgule** : les erreurs d'arrondi s'accumulent ; restez en entiers.
- **Valider seulement dans le setter** : la validation doit aussi protéger le constructeur.

10. Pour votre rapport

En plus de la structure du cadre commun, répondez à ces questions propres au sujet :

- Pourquoi avoir choisi `recharger()` et `consommer()` plutôt qu'un setter pour la batterie ? Quelle règle est ainsi protégée ?
- Quelle duplication de code la classe mère a-t-elle évitée entre trottinette, vélo et voiture ?
- Quelle est la ligne de code où le **type réel** d'un véhicule décide de son tarif ou de son alerte de panne ?
- Que faudrait-il modifier pour ajouter une moto électrique ? Combien de classes existantes ?

11. Modifications en direct possibles à la soutenance

Pendant la démonstration, on pourra vous demander l'une de ces modifications, en quelques minutes :

- Ajouter une **moto électrique** (300 millimes par minute, 1,5 % de consommation par km) et la voir apparaître dans la recette sans toucher à la station.
- Passer le seuil de disponibilité de 15 % à 20 %.
- Passer le tarif de nuit à moins 30 %.
- Ajouter un état « en maintenance » qui empêche la location.

Annexe : les liens entre les classes (indicatif)

Voici seulement les **relations** attendues à l'arrivée. Les attributs et les méthodes, c'est à vous de les concevoir.

Relation	Sens
Trottinette, VeloElectrique, VoiturePartagee → Vehicule	Héritage : chaque type **est un** véhicule.
Station → Vehicule (de 0 à plusieurs)	La station **contient** des véhicules de tous types, par le type commun.
Demo, Menu → Station	Les classes de test et de menu **utilisent** la station.